Literature review

Three Research Papers

Interactive code generation, agentic RAG for testing, and the agent harness

Lahiri et al. · Hariharan et al. · Macedo

Overview

The three papers at a glance

TiCoder

How can a user's intent be pinned down?

Yes/no answers on generated tests prune and rank code.

Agentic RAG for testing

How can agents with retrieval generate test artifacts?

Hybrid vector-graph retrieval plus five agents reach 94.8% accuracy.

What makes a harness

What makes a system an agent harness?

Four conditions: loop, tools, context and control.

Paper 1 · Lahiri et al., Microsoft Research

TiCoder: ask about tests, not code

The problem

Natural language is ambiguous, so users cannot tell whether generated code matches their intent, and must scan long lists of suggestions.

The workflow

1. An LLM generates candidate code and tests.
2. The user answers YES, NO or UNDEFINED on one test.
3. Code that contradicts the answer is pruned.
4. Output: approved tests plus ranked code.

Paper 1 · Results with Codex

TiCoder: one query lifts accuracy

70.73%

MBPP pass@1 after one query, versus 48.24% without interaction

55.28%

HumanEval pass@1 after one query, versus 30.49%

1.7 / 1.5

Average queries to find a test matching intent (MBPP / HumanEval)

Caveat: the user is simulated by a reference implementation, and no user study has been run yet.

Paper 2 · Hariharan et al., Apple

Agentic RAG: how it works

Hybrid knowledge

SingleStore vectors for semantic search, plus TigerGraph with 15 edge types such as Requires, Validates, Impacts and Covers.

Five agents

Legacy test analysis, functional change mapping, integration points, modernized test cases and compliance validation.

Context and traceability

Staged context assembly with conflict resolution, and bidirectional links from requirements to tests to results.

Paper 2 · Results on SAP migration

Agentic RAG: accuracy by stage

65.2%	78.4%	87.1%	94.8%
Basic RAG	Vector search	Hybrid RAG	Agentic system

Artifact creation time fell 85% (240 to 36 hours per phase), with 35% cost savings.

Caveat: self-reported enterprise results; the paper gives conflicting go-live figures (2 vs 16 months).

Paper 3 · Macedo

Macedo: what is an agent harness?

The runtime layer that turns a language model into an agent acting on an external environment. It must satisfy four conditions.

T1 · Agent loop

Reasoning, action and observation interleaved at runtime

T2 · Tool interface

Lets the model perceive and alter the environment

T3 · Context

Task-aware choice of what enters the context window

T4 · Control

Limits or verification that do not rely on the model's obedience

Paper 3 · Macedo

A membership test with clear boundaries

Fails at least one condition

Agent framework: composes agents, sits above a harness

SDK: building blocks, no running loop

IDE plugin: autocomplete, no loop or control

Eval harness: judges after the run

Orchestrator: fixed graph, not adaptive

Six real harnesses pass

Claude Code, Codex CLI, Aider, Cline, OpenHands and SWE-agent all share the four-part core.

They differ most in control: guardrails, permission modes, version control, human approval and sandboxing.

Wrap-up

Key takeaways, paper by paper

TiCoder

Yes/no answers on generated tests formalize user intent. One query raised MBPP pass@1 from 48.24% to 70.73%.

Agentic RAG

Graph relationships and specialized agents on top of vector retrieval raised accuracy from 65.2% to 94.8% in the authors' tests.

Macedo

An agent harness has four conditions: loop, tools, context and control. Control is where real designs differ most.

Wrap-up

Limitations, paper by paper

TiCoder

Single functions, two Python benchmarks, a simulated user and a discontinued Codex model.

Agentic RAG

One company's SAP project, self-reported results and inconsistent figures across sections.

Macedo

A definitional paper: it proposes no new agent and reports no performance measurements.